

Lecture Scribe: Dijkstra's Shortest Path Algorithm

Advanced Algorithms & Complexity

Student ID	Student Name	Student Email
2026H1030047G	Dhruv Rakeshbhai Patel	h20260047@goa.bits-pilani.ac.in
2026H1030049G	Yash Kamlesh Chitroda	h20260049@goa.bits-pilani.ac.in

1 Introduction

Dijkstra's Algorithm is a **single-source shortest path (SSSP)** algorithm used to find the minimum-cost path from one source vertex to every other reachable vertex in a **weighted graph with non-negative edge weights**. It can be applied to both directed and undirected graphs.

The central idea is greedy: at every step, choose the unprocessed vertex whose currently known distance from the source is smallest, finalize that distance, and use it to update the distances of its neighbors.

The problem can be viewed as assigning each edge a cost, such as distance, time, or network latency, and then determining the least total cost required to travel from a given source to every other vertex. For example, in a road network, vertices can represent locations and edge weights can represent distances between locations.

Dijkstra's Algorithm is particularly useful because it is considerably faster than more general shortest-path algorithms when all edge weights are non-negative. MIT's algorithms material presents Dijkstra's Algorithm specifically for weighted graphs with non-negative edge weights, while Bellman-Ford is used when negative edge weights are allowed.

2 Main Idea and Greedy Strategy

Initially, the shortest known distance to the source itself is zero, while the distance to every other vertex is considered infinity:

$$dist[source] = 0$$

$$dist[v] = \infty \quad \text{for } v \neq source$$

A set of unprocessed vertices is maintained. At each iteration:

1. Select the unprocessed vertex u with the smallest tentative distance.
2. Treat u 's current distance as final.
3. Examine all edges leaving u .
4. Try to improve the distance of every neighboring vertex.
5. Repeat until all reachable vertices have been processed.

The distance update operation is called **relaxation**:

$$dist[v] = \min(dist[v], dist[u] + w(u, v))$$

where $w(u, v)$ is the weight of the edge from u to v .

If reaching v through u gives a shorter path than the currently stored distance, the distance is updated.

The important greedy observation is that once the unprocessed vertex with the smallest tentative distance is selected, that distance cannot later become smaller when all edge weights are non-negative. Therefore, the selected vertex can be permanently marked as finalized. This property is the basis of Dijkstra's correctness.

3 Step-by-Step Example

Consider a graph with the following weighted edges:

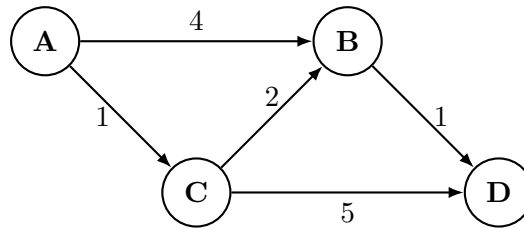


Figure 1: Weighted graph used for the Dijkstra's Algorithm example.

The graph contains the following weighted edges:

Source	Destination	Weight
A	B	4
A	C	1
C	B	2
C	D	5
B	D	1

Let A be the source.

3.1 Initialization

Vertex	Initial Distance
A	0
B	∞
C	∞
D	∞

The smallest distance is $A = 0$, so A is selected first.

3.2 Process A

From A :

$$A \rightarrow B = 4$$

$$A \rightarrow C = 1$$

Therefore:

Vertex	Distance
A	0
B	4
C	1
D	∞

The next smallest unprocessed vertex is C , with distance 1.

3.3 Process C

Through C , the distance to B is:

$$1 + 2 = 3$$

Since $3 < 4$,

$$\text{dist}[B] = 3$$

Similarly, the distance to D is:

$$1 + 5 = 6$$

Hence:

Vertex	Distance
A	0
B	3
C	1
D	6

The next smallest vertex is B , with distance 3.

3.4 Process B

From B , the distance to D becomes:

$$3 + 1 = 4$$

This improves the previous value of 6, so:

$$\text{dist}[D] = 4$$

Finally, D is selected with distance 4.

Therefore, the shortest distances from A are:

$$A = 0, \quad C = 1, \quad B = 3, \quad D = 4$$

The shortest path to D is:

$$A \rightarrow C \rightarrow B \rightarrow D$$

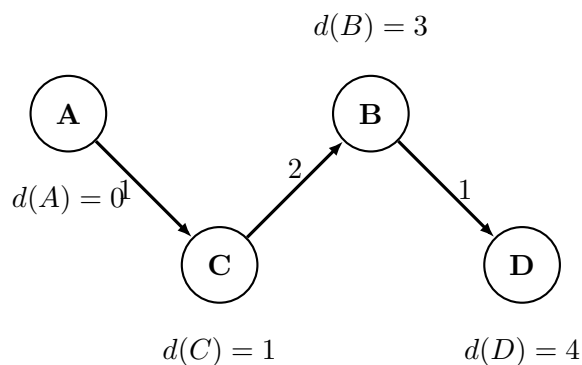


Figure 2: Final shortest-path tree after applying Dijkstra's Algorithm from source A.

The final shortest-path tree contains the edges

$$A \rightarrow C, \quad C \rightarrow B, \quad B \rightarrow D$$

with the corresponding shortest distances:

$$d(A) = 0, \quad d(C) = 1, \quad d(B) = 3, \quad d(D) = 4$$

Thus, the shortest path from A to D is:

$$A \rightarrow C \rightarrow B \rightarrow D$$

with total cost:

$$1 + 2 + 1 = 4$$

This example demonstrates the two fundamental operations in Dijkstra's Algorithm:

$$\text{Select minimum tentative distance} \quad + \quad \text{Relax adjacent edges}$$

4 Data Structures and Implementation

A basic implementation maintains:

- `dist[]` – current shortest known distance from the source.
- `visited[]` – indicates whether a vertex has been finalized.
- `parent[]` – stores the previous vertex for shortest-path reconstruction.

The simplest implementation scans all vertices to find the minimum-distance unvisited vertex. Its time complexity is approximately:

$$O(V^2 + E)$$

where V is the number of vertices and E is the number of edges.

For sparse graphs, a **priority queue (min-heap)** is generally preferred. The priority queue stores vertices according to their current tentative distances. Instead of scanning every vertex to find the smallest distance, the minimum can be extracted efficiently.

Using a binary heap gives:

$$O((V + E) \log V)$$

which is often written as:

$$O(E \log V)$$

for connected graphs.

With an adjacency-list representation, the space complexity is typically:

$$O(V + E)$$

A common priority-queue implementation is:

```
dist[source] = 0
push(source, 0) into min-heap

while heap is not empty:
    (currentDistance, u) = extract minimum

    if currentDistance != dist[u]:
        continue

    for every edge (u, v, weight):
        newDistance = dist[u] + weight

        if newDistance < dist[v]:
            dist[v] = newDistance
            parent[v] = u
            push(v, newDistance)
```

Checking for an outdated queue entry is important when a new pair is inserted into the heap each time a distance improves.

5 Why Dijkstra's Algorithm Works

The correctness of Dijkstra's Algorithm depends critically on the **non-negative edge-weight condition**.

Suppose the algorithm selects an unprocessed vertex u with the smallest tentative distance. Since all remaining edges have weight greater than or equal to zero, moving through another unprocessed vertex cannot produce a route to u with a smaller total cost than the value already assigned to u .

Consequently, once u is selected as the minimum-distance unprocessed vertex, its distance is guaranteed to be the true shortest-path distance.

The algorithm can therefore be viewed as maintaining two groups:

- **Finalized vertices:** their shortest distances are known with certainty.
- **Unfinalized vertices:** their distances are tentative and may still improve.

The algorithm gradually expands the finalized region outward from the source.

6 Important Limitation: Negative Edge Weights

Dijkstra's Algorithm **cannot, in general, be used when negative-weight edges are present.**

The greedy strategy assumes that extending a path cannot decrease its total cost. A negative edge violates this assumption. A vertex that currently appears to have the smallest distance may later be reached through another path containing a negative edge, making its true distance smaller after it has already been finalized.

Therefore:

Graph Condition	Suitable Algorithm
Unweighted / unit-weight graph	BFS
Non-negative edge weights	Dijkstra
Negative edge weights	Bellman–Ford

Dijkstra also cannot meaningfully solve a shortest-path problem containing a negative-weight cycle because repeatedly traversing the negative cycle can reduce the total path cost indefinitely.

7 Dijkstra vs. BFS and Bellman–Ford

The three algorithms solve related shortest-path problems under different assumptions.

Algorithm	Graph Type	Weights	Typical Complexity
BFS	Unweighted	Equal/unit cost	$O(V + E)$
Dijkstra	Weighted	Non-negative	$O((V + E) \log V)$
Bellman–Ford	Weighted	May be negative	$O(VE)$

BFS works because every edge effectively has the same cost, so processing vertices level by level produces the shortest paths.

Dijkstra generalizes this idea to non-uniform but non-negative edge weights by always expanding the vertex with the smallest accumulated cost.

Bellman-Ford is slower but more general because it can handle negative edge weights.

8 Applications

Dijkstra's Algorithm has several practical applications because many real-world problems can be modeled as weighted graphs.

- **Navigation and route planning:** locations are vertices and road distances or travel costs are edge weights.
- **Computer networks:** routers are vertices and communication costs are edge weights.
- **Transportation systems:** stations and connections can be represented as a weighted graph.
- **Robotics and path planning:** locations or states can be modeled as graph vertices with movement costs.
- **Resource optimization:** the algorithm can identify a minimum-cost sequence of transitions.

9 Key Takeaways

Dijkstra's Algorithm should be remembered as a:

Greedy single-source shortest-path algorithm for graphs with non-negative edge weights.

The most important points are:

1. Initialize the source distance to 0 and all others to infinity.
2. Repeatedly select the vertex with the smallest tentative distance.
3. Finalize that vertex.
4. Relax all of its adjacent edges.
5. Continue until all reachable vertices are finalized.
6. Use a min-heap or priority queue for efficient implementation on sparse graphs.
7. Store parent information when the actual shortest path is required.
8. Do not use standard Dijkstra when negative edge weights are present.

The central formula to remember is:

$$\boxed{dist[v] = \min(dist[v], dist[u] + w(u, v))}$$

In essence, Dijkstra's Algorithm repeatedly asks:

“Among all vertices whose shortest distance is not yet finalized, Which one is currently cheapest to reach?”

This greedy choice, together with the non-negative edge-weight condition, allows the algorithm to construct shortest paths efficiently.

Sources Used

1. GeeksforGeeks, “Introduction to Dijkstra's Shortest Path Algorithm,” <https://www.geeksforgeeks.org/dsa/introduction-to-dijkstras-shortest-path-algorithm/>
2. GeeksforGeeks, “Dijkstra's Shortest Path Algorithm using Set in STL,” <https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-using-set-in-stl/>
3. CP-Algorithms, “Dijkstra – finding shortest paths from a given vertex,” <https://cp-algorithms.com/graph/dijkstra.html>
4. MIT OpenCourseWare, “Lecture 13: Dijkstra,” MIT 6.006 Introduction to Algorithms, <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/lecture-13-dijkstra/>
5. Stanford University, CS106B, “Graph Algorithms / Dijkstra and A*,” <https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1258/lectures/26-graph-algorithms/>